

Move a legacy .NET Framework and Java application to Azure – with GitHub Copilot doing the work

Assess the code, get the customer's go-ahead, and let GitHub Copilot modernization do the rest end to end: modernize both applications, create the Azure resources and deploy the apps. Then prove they still work. One guide, one path, step by step.

How this guide works

Who	Does
PRESENTER	Part 1 once, before the event. Part 7 on the day.
EVERYONE	Parts 2 to 6, in order – participants, and the presenter when rehearsing.

- Every step tells you **what to do** and **what you should see**. When it matches, tick **Done**.
- **All addresses and names are already in the commands**. Click **Copy**, paste, run. Only your team name can change – see [Your lab at a glance](#).
- **In VS Code you use the modernization panel once**, to start (3.3). After that everything happens in one Copilot Chat – see [Where you work in VS Code](#).
- **Copilot does the work**. Don't change the code yourself. Read what Copilot shows you, then approve.

The scenario

Al Thuraya Holding (fictitious) runs its **Group Finance Portal** on one Windows server in its own data centre: an **ASP.NET MVC 5 portal on .NET Framework 4.8** in IIS, a **Java 8 / Spring Boot 2.7 treasury service** in Tomcat, and two **SQL Server** databases. The CFO wants it on Azure – without a rewrite project.

Before – the customer's server (the lab VM)

IIS 10

Portal · ASP.NET MVC 5 · .NET Framework 4.8

Tomcat 9

Treasury service · Java 8 · Spring Boot 2.7

SQL Server Express

Two databases · Windows authentication and a SQL password

Local disk and Windows

Bank files, reports, e-mail folder, log files, Event Log

GitHub Copilot modernization



After – Azure

App Service

Portal · .NET 10

App Service

Treasury service · Java 21 · Spring Boot 3

Azure SQL Database

Managed identities – no passwords

Blob Storage and Application Insights

Bank files and reports · monitoring

The journey

1

Prepare the lab

Presenter, once

2

Set up

Your laptop

3

Assess

Findings report

4

Decide & plan

Customer go-ahead

5

Modernize & deploy

Copilot changes the code and deploys it

6

Test

Azure, same behaviour

The tools

Tool	What it does here	Part
PowerShell 7 and Azure CLI	Run the lab scripts and the Azure commands – Copilot's deployment uses your Azure CLI sign-in	1, 5, 6
Lab scripts (<code>onprem-vm/</code>)	Create, refresh and remove the "on-premises" VM	1, 7
VS Code with <code>GitHub Copilot modernization</code> – chat agent <code>modernize</code>	Assessment, plans, code changes and the deployment to Azure	3–5
<code>AppCAT</code>	The analysis engine behind the assessment – installed automatically	3
Bicep and a deployment script – written by Copilot	Create the Azure resources and deploy both apps (<code>infra/</code>)	5
<code>Invoke-SmokeTest.ps1</code>	Proves the app behaves the same before and after	1, 3, 6
Remote Desktop	Look inside the "on-premises" server – optional	1
git	Every Copilot task becomes a commit you can review	2–5

Your lab at a glance

These are all the addresses you use. Every command in this guide already contains them.

What	Address	What it is
Today – on the lab VM, the customer's "on-premises" server		
Portal	http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com	The website the finance team uses – .NET Framework 4.8 on IIS
Treasury service	http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com:8080/treasury-service	The Java service behind the portal: bank accounts, cash and payment runs – Java 8 on Tomcat, port 8080 of the same VM
Remote Desktop	<code>contoso-onprem-0720c.swedencentral1.cloudapp.azure.com</code>	Sign-in to the VM itself – user <code>contosoadmin</code> , password from your presenter
The code		
Repository	github.com/zhshah/AlThuraya-App-Modernization-Lab	The application's source code – the same code that runs on the VM. Private: your presenter gives you access.
After Part 5 – on Azure, created by Copilot		
Portal	<code>https://app-portal-0720c-team1.azurewebsites.net</code>	The same portal on App Service, .NET 10
Treasury service	<code>https://app-treasury-0720c-team1.azurewebsites.net</code>	The same service on App Service, Java 21

Your team name

team1

Working alone? Keep `team1`. At an event, your presenter gives each team a name – `team1`, `team2`, ... – and all Azure names in this guide change with it. `0720c` is this lab's ID; it keeps the Azure names unique.

1

Prepare the lab

PRESENTER Run these from the lab kit folder (the one with `onprem-vm/`) in PowerShell 7

PowerShell 7 Azure CLI Lab scripts Key Vault Remote Desktop

The lab is one Windows Server VM in Azure that plays the customer's on-premises server: IIS, SQL Server and Tomcat on one machine, running the portal and the treasury service with demo data. Your lab VM already exists: `vm-contoso-web01` in resource group `rg-contoso-onprem-swc`.

The lab kit – this guide, the lab scripts and the application – is the private repository [zhshah/AlThuraya-App-Modernization-LabKit](https://github.com/zhshah/AlThuraya-App-Modernization-LabKit). Run Part 1 from a clone of it: `git clone https://github.com/zhshah/AlThuraya-App-Modernization-LabKit.git`, then `cd AlThuraya-App-Modernization-LabKit`.

1.1 Sign in to Azure

5 min

DO

```
winget install Microsoft.PowerShell # once - then use "PowerShell 7" for all lab commands
winget install Microsoft.AzureCLI # once
az login # choose the subscription that holds the lab when asked
az group show -n rg-contoso-onprem-swc --query location -o tsv
```

YOU SHOULD SEE

The last command prints `swedencentral`. If it says the resource group was not found, you are in the wrong subscription: run `az login` again and pick the other one. You need **Contributor** or **Owner** on that subscription.

1.2 Build the lab (first time only)

35–50 min

Your lab already exists – skip this step. Run it only if the lab was removed and you need to build it again:

```
./onprem-vm/Deploy-Lab.ps1 -ResourceGroup rg-contoso-onprem-swc -VnetResourceGroup sweden-central-vnet -VnetName Sweden-vNet -SubnetName default
```

WHAT IT DOES

Creates the VM in your existing network `Sweden-vNet` (open only to your internet address), installs IIS, SQL Server Express, Java 8 and Tomcat, builds and deploys both applications with demo data, and tests everything. It reuses the same names, so the addresses in this guide stay valid.

It stopped with an error? Run the same command again – it continues where it stopped.

YOU SHOULD SEE

At the end it prints the portal, treasury and Remote Desktop addresses – the same as in [Your lab at a glance](#).

1.3 Check that it works

5 min

DO

1. Open the portal: <http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com>
2. Run the test. It needs the two addresses of the app – the portal and the treasury service:

```
$portal = 'http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com'
$treasury = 'http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com:8080/treasury-service'
./scripts/Invoke-SmokeTest.ps1 -PortalUrl $portal -TreasuryUrl $treasury
```

3. Try Remote Desktop. The first line copies the VM password to your clipboard; sign in as `contosoadmin` and paste it:

```
az keyvault secret show --vault-name kv-contoso-onprem-0720c -n vm-admin-password --query value -o tsv | Set-Clipboard
mstsc /v:contoso-onprem-0720c.swedencentral.cloudapp.azure.com
```

YOU SHOULD SEE

The portal opens, the test ends with `SMOKE_TEST_PASSED`, and you can sign in to the VM.

1.4 Prepare the participants

before the event

- ❑ **Azure:** for each team a resource group – `rg-finance-team1`, `rg-finance-team2`, ... – in which the team is **Owner**: Copilot's deployment gives the apps' managed identities their roles. Check that App Service (Linux, B1), Azure SQL and Communication Services are available in Sweden Central.
- ❑ **Azure Policy:** does a policy in that subscription switch off public network access? Add your exemption to your prompt in 4.2, for example *Tag every resource SecurityControl=Ignore*. – our test did this.
- ❑ **GitHub Copilot:** a licence with premium requests for everyone, with agent mode allowed by their organization.
- ❑ **Code:** add every participant's GitHub account to [zhshah/AlThuraya-App-Modernization-Lab](#) (Settings → Collaborators; read access is enough).
- ❑ **Send them this guide** – `lab-guide.html` together with its `images` and `sample-plans` folders – and ask them to finish Part 2 before the event.
- ❑ **On the day, give them** their team name – and the VM password (from 1.3) if they want to look inside the server.

1.5 On the event day

10 min

DO – CONNECTED TO THE VENUE NETWORK

```
$myIp = (Invoke-RestMethod https://api.ipify.org).ToString().Trim() + '/32'
foreach ($rule in 'Allow-RDP-From-Presenter', 'Allow-HTTP-From-Presenter', 'Allow-Tomcat-From-Presenter') {
    az network nsg rule update -g rg-contoso-onprem-swc --nsg-name nsg-vm-contoso-web01 -n $rule --source-address-
    prefixes $myIp -o none
}
"VM open to $myIp"
./onprem-vm/Update-LabApp.ps1 -ResourceGroup rg-contoso-onprem-swc -ResetDemoData
```

WHAT IT DOES

The VM only accepts connections from allowed internet addresses. The first lines allow the venue's address – so everyone on the venue network can open the portal. The last line puts fresh demo data and the latest scripts on the VM (3–5 minutes) and starts the VM if it was stopped.

The site doesn't load? Almost always the network changed. Run the first four lines again from the network you are on.

YOU SHOULD SEE

VM open to ..., then `SMOKE_TEST_PASSED` at the end of the update.

2 Set up your laptop

EVERYONE About 30 minutes · do it before the event – the downloads are large

winget

VS Code

GitHub Copilot modernization

.NET 10 SDK

Build Tools

git

You need **Windows 10 or 11**, a **GitHub account with GitHub Copilot** (added to the lab repository by your presenter), and **Owner** rights on your team's Azure resource group `rg-finance-team1`.

2.1 Install the tools

20 min

DO – IN POWERSHELL; ACCEPT THE WINDOWS PROMPTS

```
winget install Microsoft.VisualStudioCode
winget install Microsoft.DotNet.SDK.10
winget install Git.Git
winget install Microsoft.AzureCLI
winget install Microsoft.Azd
winget install OpenJS.NodeJS.LTS
winget install Microsoft.VisualStudio.2022.BuildTools --override "--quiet --wait --add
Microsoft.VisualStudio.Workload.WebBuildTools;includeRecommended --add Microsoft.Net.Component.4.8.TargetingPack"
```

Then **open a new PowerShell window** and add the VS Code extensions:

```
code --install-extension vscjava.migrate-java-to-azure
code --install-extension ms-dotnettools.upgrade-agent
code --install-extension ms-dotnettools.csdevkit
```

GOOD TO KNOW

- The Build Tools are needed to assess the .NET Framework project. Already have Visual Studio 2022 or 2026 with *ASP.NET and web development*? Skip that line.
- You don't need Java: Copilot downloads Java 21 and Maven when it needs them.
- Node.js: Copilot's deployment used it to load the demo data into Azure SQL.

2.2 Get the code

5 min

DO – IN A NEW POWERSHELL WINDOW

```
mkdir C:\Lab -Force
cd C:\Lab
git clone https://github.com/zhshah/AlThuraya-App-Modernization-Lab.git finance-portal
cd finance-portal
git checkout -b modernize/team1
code .
```

1. The first time, git asks you to sign in to GitHub – use the account your presenter added.
2. `code .` opens the folder `C:\Lab\finance-portal` in VS Code. When VS Code asks whether you trust the authors, choose **Yes, I trust the authors** – the agents don't run otherwise.
3. Sign in to GitHub Copilot: Accounts icon, bottom left.
4. Open the VS Code terminal (**Terminal** → **New Terminal**) and sign in to Azure: `az login`. Use this terminal for all the following commands.

WHAT YOU GET

The customer's application exactly as it runs on the lab VM, in `C:\Lab\finance-portal`: `src\dotnet` (portal), `src/java/treasury-service`, `database` and `scripts`.

YOU SHOULD SEE

The VS Code Explorer shows **FINANCE-PORTAL** with `src`, `database` and `scripts`. Work in this window for the rest of the lab: GitHub Copilot modernization works on the folder that is open in VS Code.

Closed VS Code? Open the folder again: **File** → **Open Folder...** → `C:\Lab\finance-portal`.

Never push to `main` – it must stay unchanged for the next session. To start over, delete the `C:\Lab\finance-portal` folder and clone again.

2.3 Check your laptop

2 min

DO

```
./scripts/Test-LabWorkstation.ps1
```

WHAT IT DOES

Checks the tools, the sign-ins and the internet access the lab needs. It changes nothing.

YOU SHOULD SEE

The last line is `WORKSTATION_READY`. If it lists a `FAIL`, follow the fix it prints, then run it again.

On the day, use a stable network: in Part 5 Copilot works for almost 4 hours and needs the internet the whole time.

3

Assess the application

EVERYONE

About 30 minutes

GitHub Copilot modernization

modernize agent

AppCAT

Invoke-SmokeTest.ps1

Where you work in VS Code

You use the GitHub Copilot modernization panel **once**, to start. After that, everything happens in **one Copilot Chat** – you don't go back to the panel.

Place	How you get there	What you do there
GitHub Copilot modernization panel	Its icon in the left bar → QuickStart	Once: click Migrate to Azure (3.3)
Copilot Chat – the chat <i>Migrate application to Azure</i>	Opens by itself in 3.3. Closed it? Open the Chat, click ← at its top and choose <i>Migrate application to Azure</i> .	Everything else: answer Copilot's questions (3.3), ask for the plans (4.2), run them and deploy (5.1)
Assessment Report tab	Opens by itself next to your files when the assessment is done	Read the findings (3.4)
Explorer – folder <code>.github/modernize</code>	The top icon in the left bar	The saved reports, the plans and the live progress of every task (What Copilot writes where)
Terminal	Terminal → New Terminal	The smoke test and the git, dotnet and az commands

Settings at the bottom of the chat box

Setting	Choose	Why
Agent – shows <code>modernize</code>	modernize	The GitHub Copilot modernization agent; Migrate to Azure selects it. If it shows <i>Ask</i> or <i>Plan</i> , click it and choose modernize . <i>Agent</i> works too – our test planned and ran the plans from a chat with <i>Agent</i> , using the same modernization tools.
Model – next to it	The newest Claude Sonnet , for example <i>Claude Sonnet 5.5</i> . Not in your list: Auto .	Microsoft recommends Claude Sonnet models for GitHub Copilot modernization. Claude Opus works too – our test used Claude Opus 5.5 – but uses more premium requests. Keep the values shown next to the model, such as <i>Max · 1M</i> (thinking level and context size).
Mode – under the box: <i>Interactive, Plan</i> or <i>Autopilot</i>	Interactive	Copilot works step by step and asks when it needs you. With <i>Plan</i> , Copilot first shows a Review Plan card and waits – click Implement with Autopilot to start (our test ran Part 5 this way).
Permissions – under the box	Default permissions ; in Part 5 Allow all	With Default, Copilot asks before it runs a command – click Allow . Part 5 runs for hours: Allow all lets Copilot run commands without waiting for you.

3.1 Look at the running app

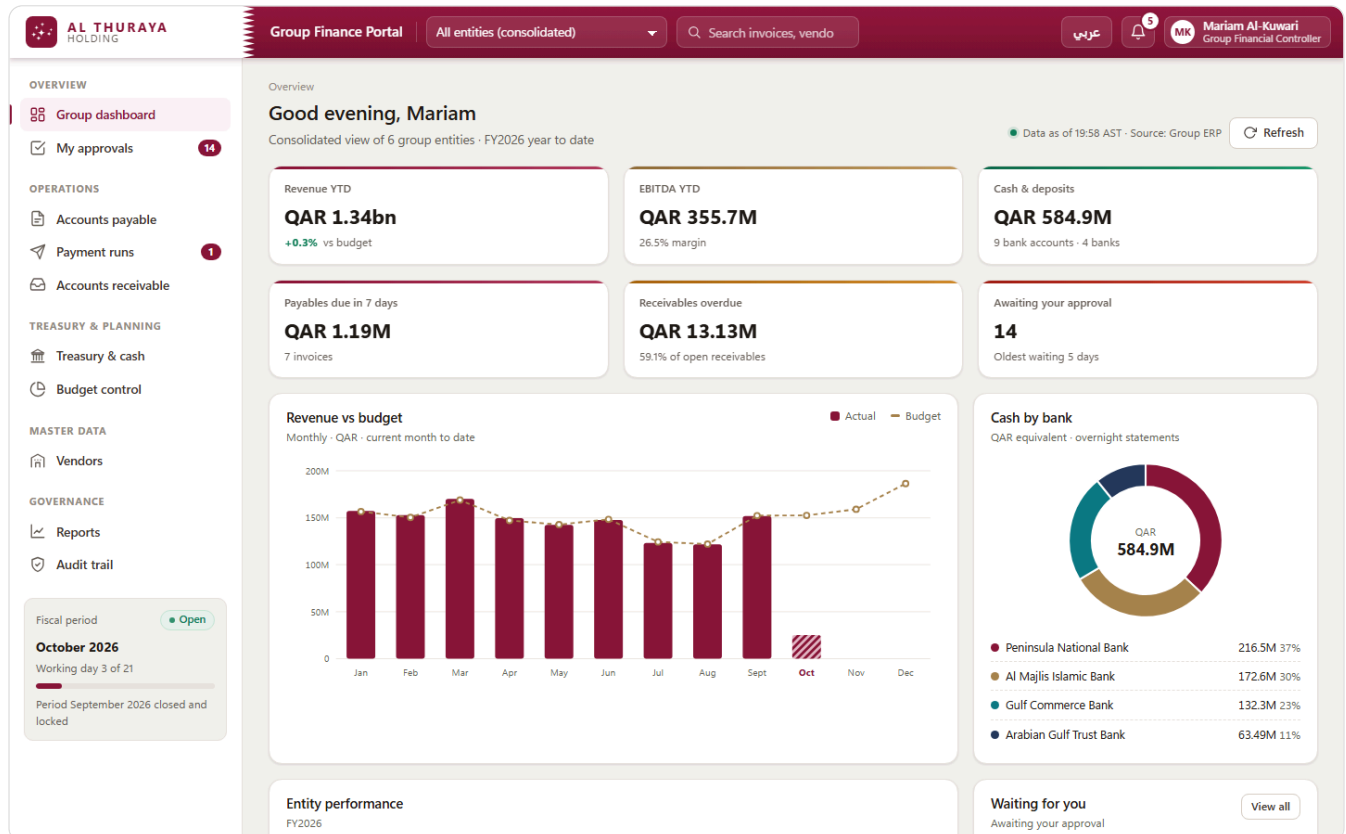
5 min

The application has two parts, both running on the lab VM. Open each one:

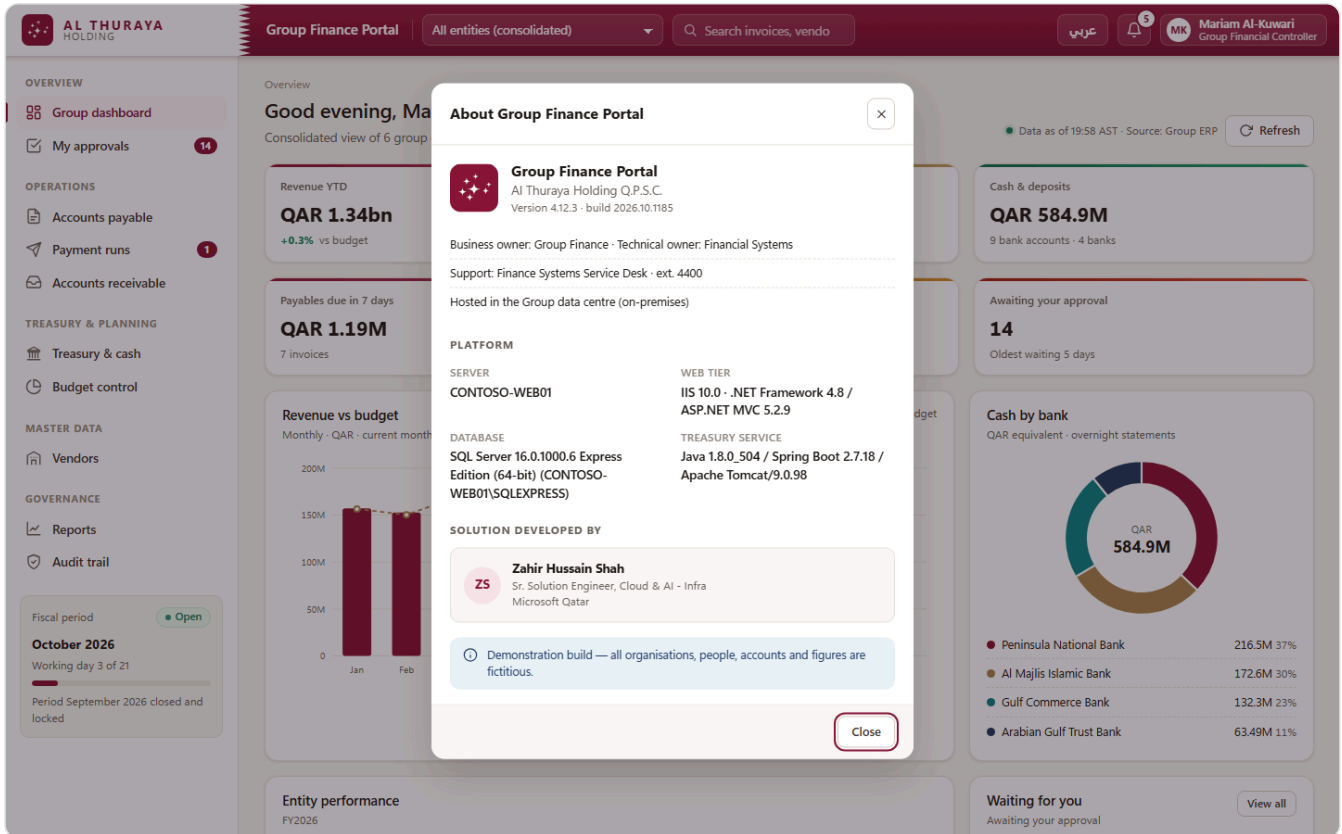
1. **The portal** – the website the finance team uses: <http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com>
Under **My approvals**, approve an invoice. User menu → **About** shows Windows Server, IIS, .NET Framework 4.8, SQL Server Express, Java 8 and Tomcat 9.
2. **The treasury service** – the Java service the portal calls for bank accounts, cash and payment runs: <http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com:8080/treasury-service>

YOU SHOULD SEE

Both pages open, and the portal shows data.



The portal today: the **Group dashboard** with demo data for the six group entities.



User menu → **About**: the platform before the migration. In Part 6 the same dialog shows .NET 10 with ASP.NET Core, Azure SQL and Java 21.

3.2 Record the baseline test

2 min

DO – IN THE VS CODE TERMINAL

```
$portal = 'http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com'
$treasury = 'http://contoso-onprem-0720c.swedencentral.cloudapp.azure.com:8080/treasury-service'
./scripts/Invoke-SmokeTest.ps1 -PortalUrl $portal -TreasuryUrl $treasury
```

THE TWO ADDRESSES

- `$portal` – the **portal**, the website you opened in 3.1.
- `$treasury` – the **treasury service**, the Java service behind the portal (port 8080 on the same VM).

WHAT IT DOES

Checks both parts: the treasury service answers, the portal page loads with its data, and the portal correctly refuses invalid requests – without changing anything. In 6.1 you run the same test against the two Azure addresses: if it passes there too, the migrated app behaves like the original.

YOU SHOULD SEE

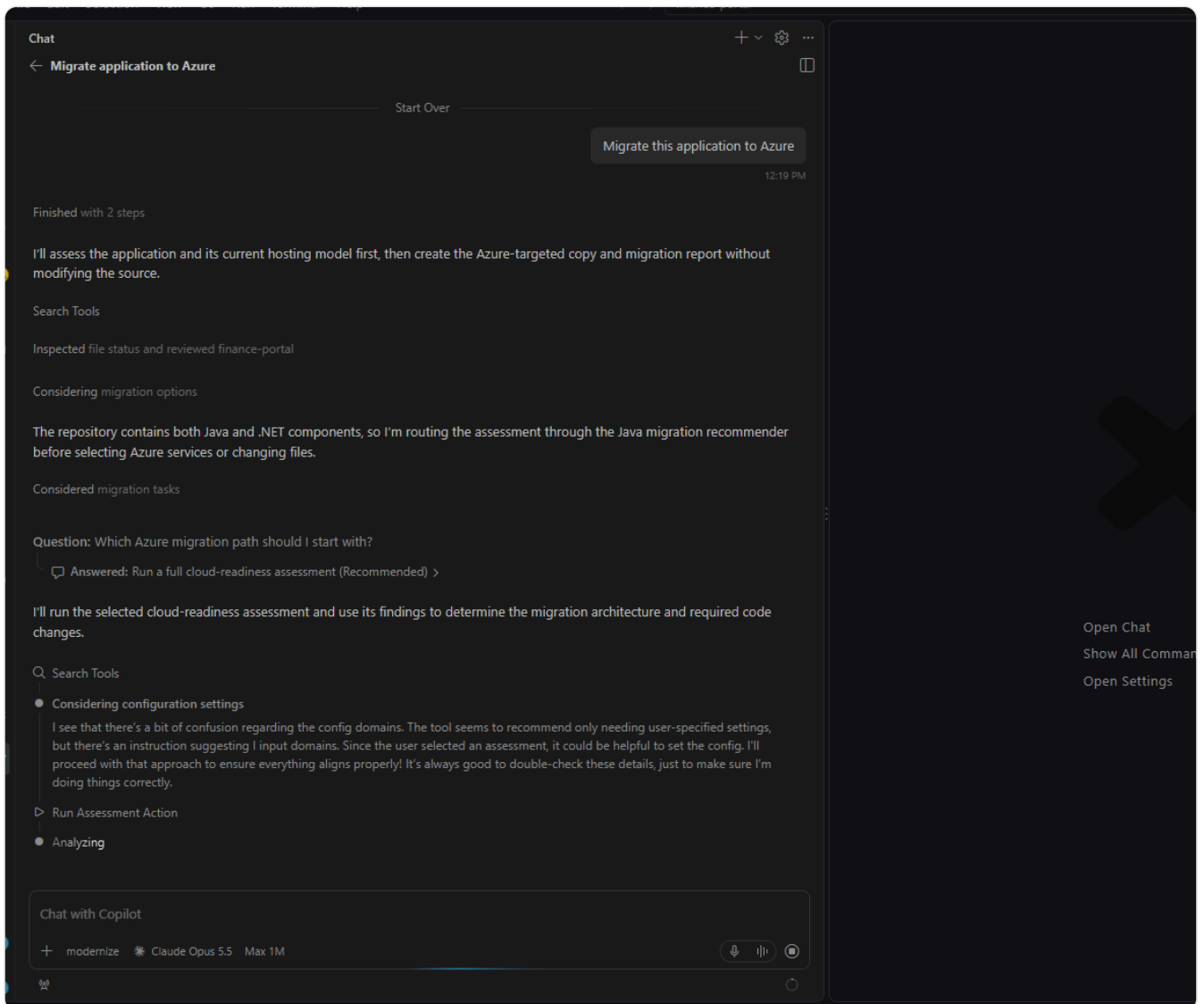
15 × `[PASS]` and `SMOKE_TEST_PASSED`.

3.3 Run the assessment

10 min

DO – IN THE VS CODE WINDOW WITH `C:\LAB\FINANCE-PORTAL` OPEN (2.2)

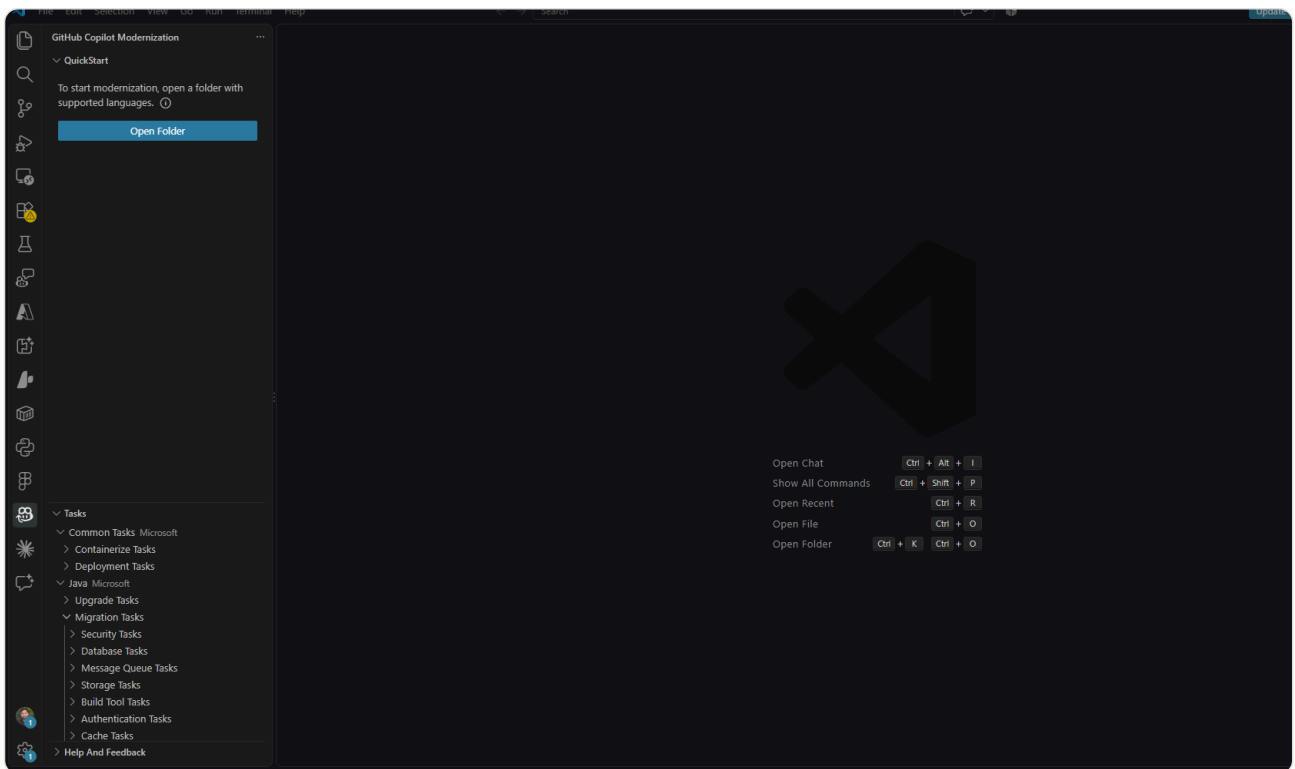
1. In the left bar, click the **GitHub Copilot modernization** icon (hover over the icons to see their names).
2. At the top of the panel, click **QuickStart** to open it. It asks *How would you like to modernize your multi-language app?*
3. Click **Migrate to Azure**. A new chat, **Migrate application to Azure**, opens and sends *Migrate this application to Azure* for you. Check the settings at the bottom of the chat box – see [Where you work in VS Code](#).
4. Copilot asks how to start. The wording changes from run to run, for example *Which Azure migration path should I start with?* Choose the option that assesses the whole application, marked **(Recommended)** – in our test **Run a full cloud-readiness assessment (Recommended)**.
5. Wait a few minutes. The first run installs the analysis tools – click **Allow** when Copilot asks. The **Assessment Report** tab opens.
6. Copilot asks its next question, for example *Which Azure hosting target should I use for the migration?* or *Proceed to planning?* **Stop here – don't answer it yet.** That is the customer's decision (Part 4).



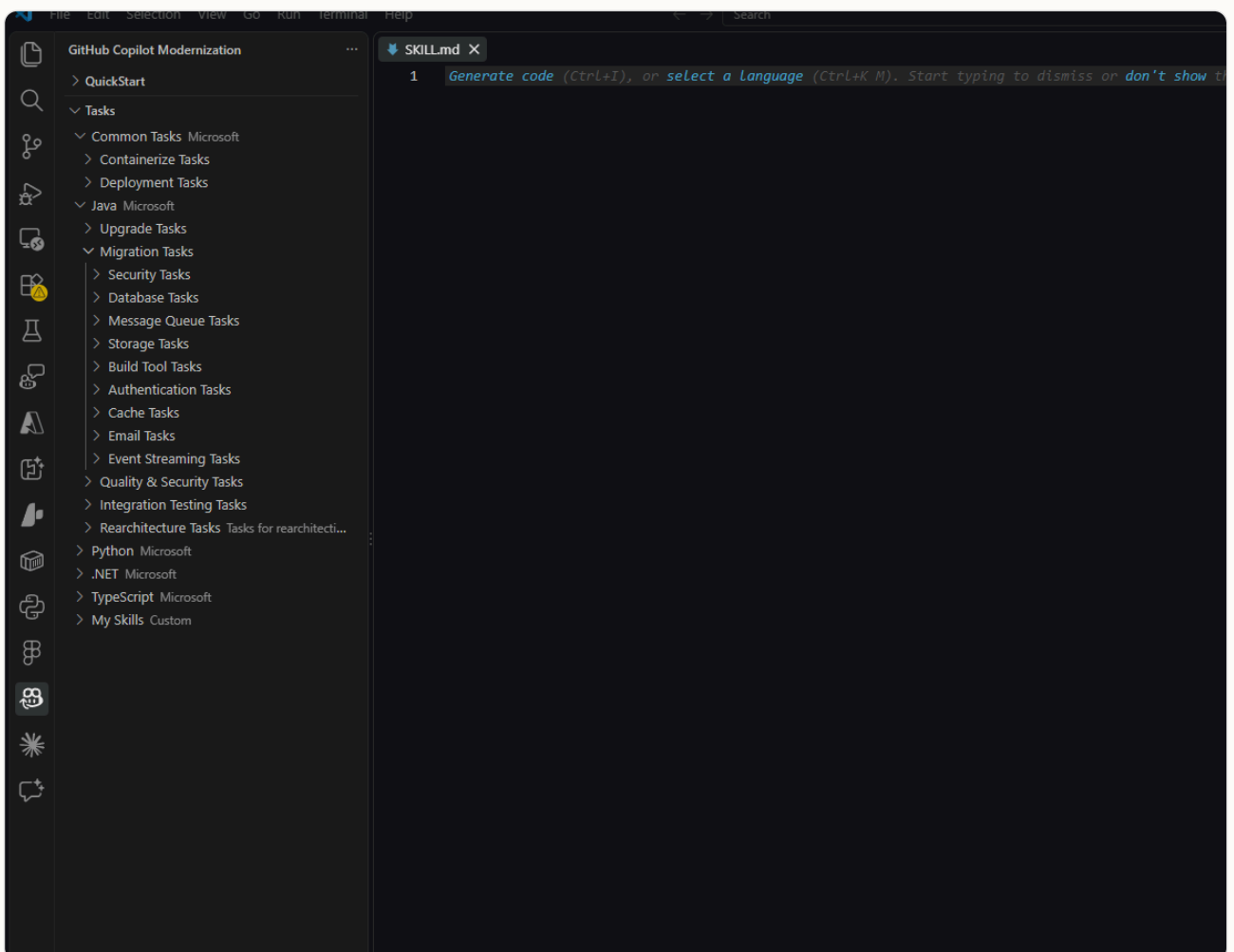
The chat after **Migrate to Azure**: the first question answered, the assessment running. At the bottom: agent *modernize* and the model.

No Migrate to Azure button? QuickStart shows only *To start modernization, open a folder...* and **Open Folder** when VS Code has no folder open. Click **Open Folder** and choose `C:\Lab\finance-portal` – the buttons appear.

Click exactly **Migrate to Azure**. The other buttons (such as **Start Assessment**) and the **Tasks** list below QuickStart start other flows that this guide doesn't follow.



No folder open: QuickStart shows only **Open Folder**. The **Tasks** list below is not used in this lab.



QuickStart collapsed (top): only the **Tasks** list shows, without **Migrate to Azure**. Click **QuickStart** to open it.

WHAT IT DOES

AppCAT, the analysis engine, scans the code for everything that blocks a move to Azure: old runtimes, resources on the local server, passwords. This standard assessment fills the report's **Issues** tab – what the plan needs.

YOU SHOULD SEE

The **Assessment Report** tab (*Report_* plus date and time), a summary of the findings in the chat, and Copilot's question – still unanswered.

The screenshot displays the AppCAT interface. On the left is a chat window titled 'Migrate application to Azure' with a 'Migrate this application to Azure' button. The chat history shows a conversation about migration options, with a question about the Azure migration path and an answer recommending a full cloud-readiness assessment. Below the chat is a 'Chat with Copilot' section with a text input and a 'Submit' button. On the right is the 'Assessment Report' for 'Report_20261005122146', created on 05/10/2026. The report includes 'Application Information' (finance-portal, Maven, Spring Boot, v4.8), a 'Cloud Readiness' donut chart showing 4 issues, and a table of issues. The table has columns for 'ISSUE CATEGORY', 'CRITICALITY', and 'STORY POINT'. The issues listed are: 'Password found in configuration file' (Potential, 3), 'Microsoft SQL database found' (Potential, 5), and 'Local JDBC Calls' (Mandatory, 5). The right sidebar contains a 'Claude Code' section with a 'Claude.ai Subscription' button and links to 'Anthropic Console' and 'Bedrock, Foundry, or Vertex'.

Chat

Migrate application to Azure

Migrate this application to Azure

Considering migration options

The repository contains both Java and .NET components, so I'm routing the assessment through the Java migration recommender before selecting Azure services or changing files.

Considered migration tasks

Question: Which Azure migration path should I start with?

Answered: Run a full cloud-readiness assessment (Recommended)

I'll run the selected cloud-readiness assessment and use its findings to determine the migration architecture and required code changes.

Evaluating migration options

The assessment found five Azure-readiness issues: local SQL access, a plaintext database password, missing containerization, Azure Container Apps configuration conflicts, and SQL Server migration work. I'm now mapping both application components so I can recommend one concrete Azure target.

Reviewed 4 files and created migration todos

Waiting for answer...

Which Azure hosting target should I use for the migration? The lowest-risk option is two Azure App Services (Windows for ASP.NET Framework and Linux/Tomcat for Spring Boot) with Azure SQL, because Azure Container Apps would require first modernizing the .NET Framework portal or using unsupported Windows containers.*

- 1 Azure App Service + Azure SQL (Recommended)
- 2 Modernize first, then use Azure Container Apps + Azure SQL
- 3 Azure Kubernetes Service + Azure SQL
- 4 Enter custom answer

Ctrl+Enter to submit Submit

Chat with Copilot

+ modernize Claude Opus 5.5 Max 1M

Assessment Reports Assessment Report

Report_20261005122146

Created on: 12:21 PM, 05/10/2026

Create Plan (4) Share Feedback

Application Information

Application Name : finance-portal Build Tools : Maven

Java Version : Unknown Frameworks : Spring Boot, Spring, v4.8

Effort : 5 (total story points: 16)

Issues Architecture API Contracts Configuration Business Workflows Dependencies Data Model

Target Service Azure Kubernetes Service (AKS)

Issue Summary

Cloud Readiness 4 issues

CRITICALITY: Mandatory Potential Optional

Domain Criticality Clear all Table Columns (3)

Cloud Readiness

ISSUE CATEGORY	CRITICALITY	STORY POINT
Password found in configuration file	Potential (1)	3
Microsoft SQL database found	Potential (1)	5
Local JDBC Calls	Mandatory...	5

Azure Table Service Azure Queue Service Azure Blob Service

Claude Code

Claude Code can be used with your Claude subscription or billed based on API usage through your Console account.

How do you want to log in?

Claude.ai Subscription

Use your Claude Pro, Team, or Enterprise subscription

Anthropic Console

Pay for API usage through your Console account

Bedrock, Foundry, or Vertex

Instructions on how to use API keys or third-party providers.

Left: Copilot's open question – leave it until 4.2. Right: the **Assessment Report**, still on *Azure Kubernetes Service (AKS)*.

3.4 Read the report

10 min

DO – IN THE ASSESSMENT REPORT TAB

1. **Target Service** shows *Azure Kubernetes Service (AKS)*. Change it to **Azure App Service** – our target. The list then shows only what matters for App Service.
2. On the **Issues** tab, start with the **Mandatory** findings – they block the move. Click › next to a finding to see the file, the line and the suggested fix.
3. Don't click **Create Plan** – you plan in the chat in Part 4, after the customer's decision.

YOU SHOULD SEE

For Azure App Service, 3 findings – all in the treasury service (our test, effort S):

- **Local JDBC Calls** – *Mandatory*: the service connects to a SQL Server on the same machine.
- **Password found in configuration file** – *Potential*: the database password in `application.properties`.
- **Microsoft SQL database found** – *Potential*: the database moves too.

With AKS selected you see 4 (plus *No Dockerfile found*), with Container Apps 5. Numbers can change with newer tool versions.

Only the treasury service in the report? That's how it ran in our test: for a repository with Java and .NET code, the assessment analyses the Java service. Copilot reads the portal's .NET code itself when it writes the plans (4.2), and every portal blocker – Windows authentication, Event Log, e-mail folder, local paths – becomes a task. The portal's findings from a separate .NET assessment are under *All findings* below.

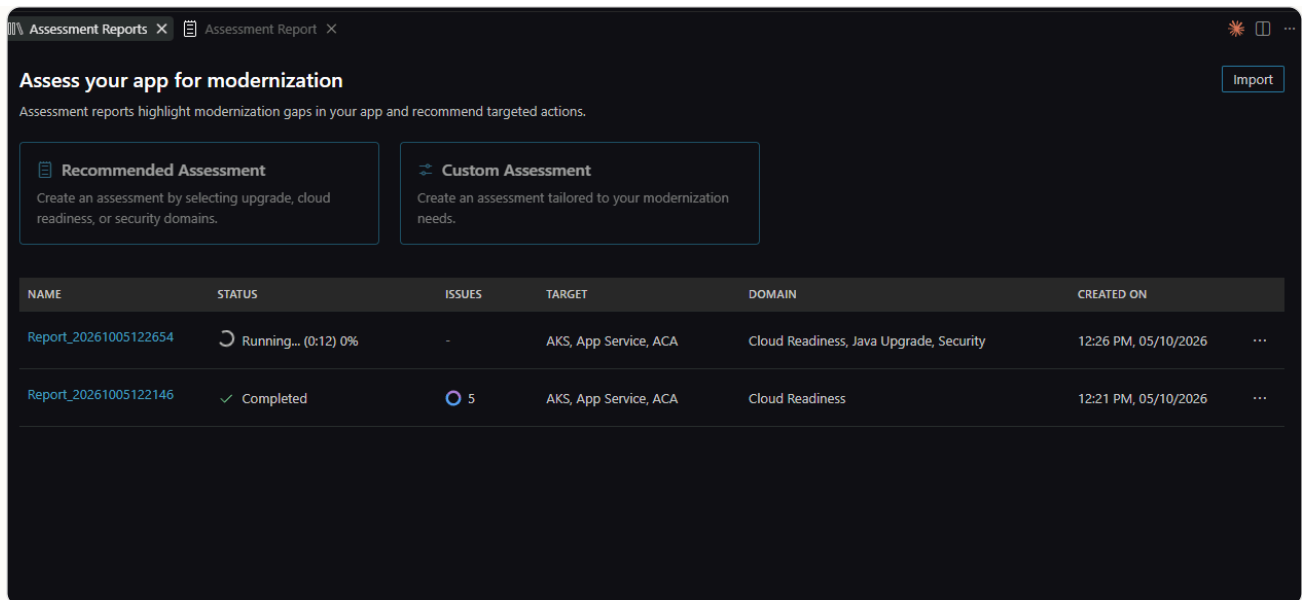
Why are Architecture, API Contracts, Configuration, Business Workflows, Dependencies and Data Model empty? The standard assessment is *Issue only*: it fills the **Issues** tab. The six other tabs are 6 AI deep-analysis documents – architecture, dependencies, API contracts, data model, configuration and business workflows. Only a **Custom Assessment** with **Full analysis** generates them, and that takes much longer – 30 minutes and more. The plan doesn't need them: **the standard assessment does the job**.

Where is the report saved? In `.github/modernize/assessment/` – one folder `reports/report-<date and time>` per report. Open it again any time from the **Assessment Reports** tab. Details: [What Copilot writes where](#).

▼ Optional – the full assessment (Custom Assessment)

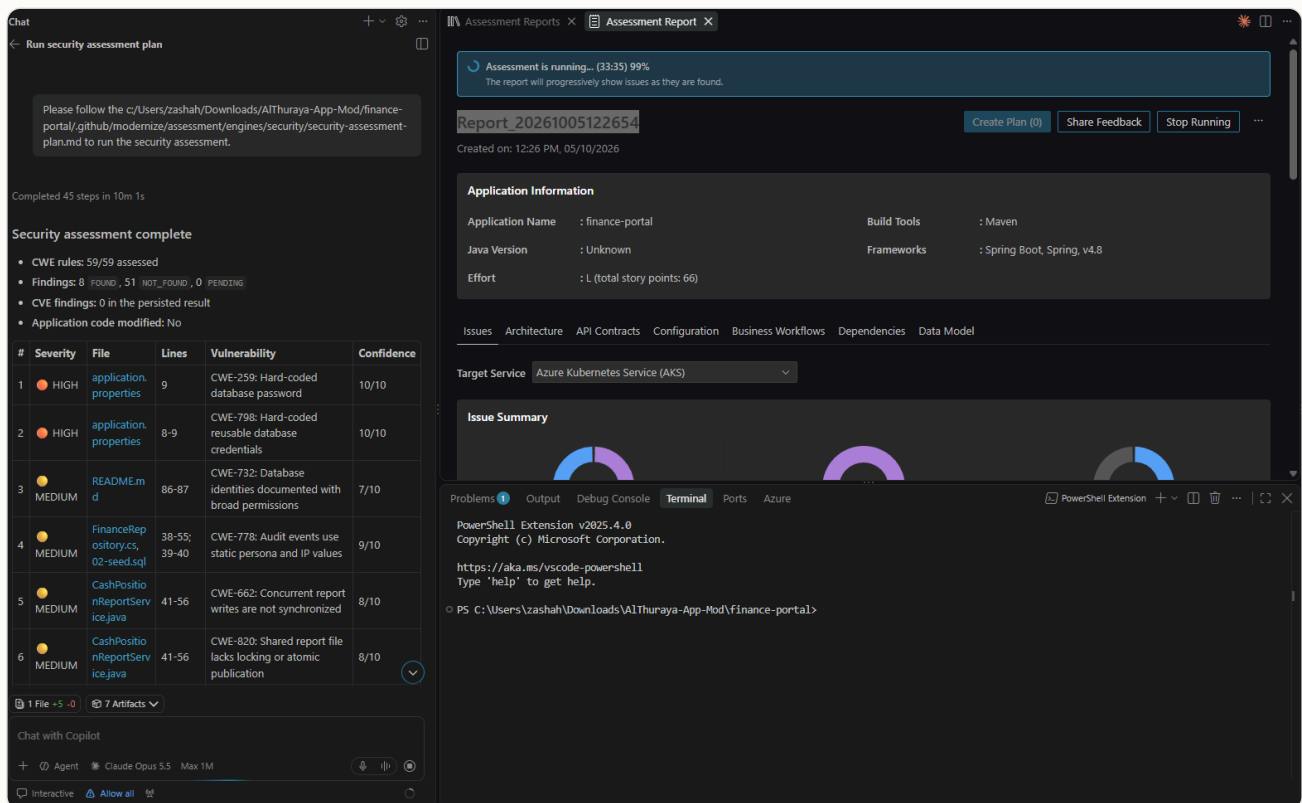
Not needed for the lab. Use it to get the six extra tabs and a security scan – for example to prepare a customer demo the day before.

1. Open the **Assessment Reports** tab (next to *Assessment Report* – it says *Assess your app for modernization*) and click **Custom Assessment**.
2. Choose **Assessment Domains**: Java Upgrade, Cloud Readiness and Security. **Analysis Coverage**: Full analysis. **Target Runtime**: OpenJDK 21. **Target Compute Services**: Azure App Service. **Target Operating System**: Linux.
3. Click **Run Assessment**. Copilot opens extra chats for the security scan and the six documents, for example *Run security assessment plan*. Let them work, and do the lab steps in the *Migrate application to Azure* chat.



The **Assessment Reports** tab: **Custom Assessment** at the top, every report below.

In our test it added *Java Version Has Reached the End of Support* and 8 security findings (CWE, for example the hard-coded database password): 14 issues, effort L. The six documents hadn't arrived after more than 30 minutes and the report stayed at 99%. If that happens, click **Stop Running** – the findings stay in the report.



Left: the security scan's results in its own chat. Right: the custom report at 99% – **Stop Running** is at the top right.

Security findings are only reported here. Copilot fixes them only if your plan request asks for it – see the two scopes in 4.2.

▼ All findings from our test runs

Treasury service (Java) – VS Code assessment, 5 October 2026

Finding	Criticality	Where
Local JDBC Calls	Mandatory	application.properties
Password found in configuration file	Potential	application.properties

Finding	Criticality	Where
Microsoft SQL database found	Potential	<code>application.properties</code>
No Dockerfile found	Mandatory – AKS and Container Apps only	–
Restricted configurations found	Potential – Container Apps only	<code>application.properties</code>
Java Version Has Reached the End of Support	Mandatory – full assessment only	<code>pom.xml</code>

The full assessment's security scan also found 8 CWE weaknesses, for example *CWE-259 Use of Hard-coded Password* and *CWE-798 Use of Hard-coded Credentials* in `application.properties`.

Group Finance Portal (.NET) – .NET assessment, 2 October 2026: 15 issues, 92 occurrences

Finding	Criticality	Where
Windows authentication detected (3)	Mandatory	<code>Web.config</code>
Windows local system features usage (2)	Mandatory	<code>SystemStatusService</code>
Hard-coded local or network paths	Mandatory	<code>PortalSettings</code>
Logging to local or network paths	Mandatory	<code>Web.config</code>
SMTP connections detected (4)	Potential	<code>EmailNotifier</code>
Access to external resources via HTTP (3)	Potential	<code>TreasuryServiceClient</code>
Local OS environment access (3)	Potential	<code>Global.asax.cs</code>
Hard-coded URLs	Potential	<code>PortalSettings</code>
Local or network I/O operations	Potential	<code>HomeController</code>
SQL database connection	Potential	<code>Web.config</code>
<code>System.Data.SqlClient</code> dependency (66)	Optional	Data access code
Connection strings without configuration builders	Optional	<code>Web.config</code>
Synchronous API usage	Optional	<code>PortalDataService</code>
Static content	Optional	Project
Writing to the Windows Event Log	Optional	<code>PortalLog</code>

4

Decide and plan

EVERYONE About 30 minutes · the presenter plays the customer

modernize agent

4.1 Get the customer's decision

10 min

Copilot is waiting for your answer in the chat. In a real project you now go back to the customer. In 3 minutes, tell the "CIO" (your presenter) what you found, what it means, and what you recommend:

Today	On Azure
IIS + .NET Framework 4.8	App Service, .NET 10
Tomcat + Java 8	App Service, Java 21 / Spring Boot 3
SQL Server with Windows authentication and a password	Azure SQL Database with managed identities – no passwords
Bank files and reports on the local disk	Blob Storage
Event Log and log files	Application Insights
E-mail through a local SMTP folder	Communication Services Email

YOU SHOULD SEE

A decision. **No-go:** stop here – the report is the deliverable. **Go:** agree how much Copilot should change – scope **A – Essentials** or **B – Complete** (4.2) – and continue.

4.2 Ask Copilot for the plans

15 min

The plan request decides how much Copilot changes. Choose one of two scopes:

	A – Essentials lab default	B – Complete
Copilot changes	Only what the apps and their databases need to run on Azure: .NET 10, Java 21, Azure SQL, Blob Storage, e-mail, monitoring – then it deploys	The same, plus every other finding – security (CWE), potential and optional ones – and all recommendations, then it deploys
Security	The CVE task that Copilot adds to every plan, kept to patch updates. Other security findings become follow-ups.	All security findings fixed. Run the Custom Assessment with Security first (3.4), so they are in a report.
Time	About 3¼ hours in our test, from plans to live on Azure	Longer – every extra finding is extra code and tests (not timed)

What took the time in our test? Not security. The CVE task took 3 minutes for the portal (no CVEs) and 8 minutes for the treasury service (29 CVEs, fixed with patch updates). The baselines and integration tests took about an hour, the upgrades and Azure changes about an hour, and the deployment about 1¼ hours. The 8 security findings of the Custom Assessment weren't fixed, because the plan request didn't ask for it – the hard-coded password disappeared anyway with the move to managed identities.

DO – IN THE SAME CHAT, MIGRATE APPLICATION TO AZURE

Copy prompt A – or B – and give it to Copilot as the answer to its open question from 3.3:

- **A question box with options** (as in our test): click **Enter custom answer**, paste, press **Ctrl+Enter**. Don't pick one of the options – only this text asks for the .NET 10 upgrade.
- **A question in the chat text**, for example *Proceed to planning?*: paste into the chat box at the bottom and press **Enter**.

PROMPT A – ESSENTIALS

Yes, the customer approved the migration. Target: Azure App Service on Linux for both applications, with Azure SQL Database. Scope: only the changes the applications and their databases need to run on Azure. Don't fix other assessment findings, such as the security (CWE) findings – list them as follow-ups. Keep the CVE task to patch-level updates.

Write one plan per application and stop for my review – don't execute anything yet.

1. Group Finance Portal (src/dotnet): upgrade from .NET Framework 4.8 to .NET 10 (ASP.NET Core MVC); remove the Windows-only APIs (registry, Windows identity, Event Log) so it runs on Linux App Service; use Azure SQL Database with passwordless managed identity authentication instead of Windows authentication; replace the SMTP pickup-folder e-mail with Azure Communication Services Email; replace the Windows Event Log and local log files with OpenTelemetry and Application Insights; read the treasury service URL from configuration.
2. Treasury service (src/java/treasury-service): upgrade to Java 21 and Spring Boot 3; use Azure SQL Database with passwordless managed identity authentication; move the bank files and cash-position reports from the local disk to Azure Blob Storage with managed identity.
3. Deployment, as the last task of each plan – the portal plan runs first: deploy both applications into the existing resource group rg-finance-**team1** in swedencentral. One Linux App Service plan, plan-finance-**team1** (B1). Web apps app-portal-0720c-**team1** (.NET 10) and app-treasury-0720c-**team1** (Tomcat 10.1, Java 21, the WAR at the site root). Azure SQL server sql-finance-0720c-**team1** with Microsoft Entra-only authentication and the databases ThurayaFinance and ThurayaTreasury (Basic): create the schemas, load the demo data from the database folder and create database users for both managed identities. Storage account stfinance0720c**team1** with the containers the treasury service needs. Application Insights for the portal. Azure Communication Services Email with an Azure-managed domain. System-assigned managed identities, each with only the roles it needs – no passwords or keys. Public access is fine: this is a demo. At the end, run scripts/Invoke-SmokeTest.ps1 against both Azure URLs and show me the URLs.

▼ Prompt B – Complete: all findings, security included

PROMPT B – COMPLETE

Yes, the customer approved the migration. Target: Azure App Service on Linux for both applications, with Azure SQL Database. Scope: everything – fix all findings of the assessment reports, security and non-security (mandatory, potential and optional), apply all their recommendations, and fix all CVEs in the dependencies. Add a task for each group of findings. Write one plan per application and stop for my review – don't execute anything yet.

1. Group Finance Portal (src/dotnet): upgrade from .NET Framework 4.8 to .NET 10 (ASP.NET Core MVC); remove the Windows-only APIs (registry, Windows identity, Event Log) so it runs on Linux App Service; use Azure SQL Database with

passwordless managed identity authentication instead of Windows authentication; replace the SMTP pickup-folder e-mail with Azure Communication Services Email; replace the Windows Event Log and local log files with OpenTelemetry and Application Insights; read the treasury service URL from configuration.

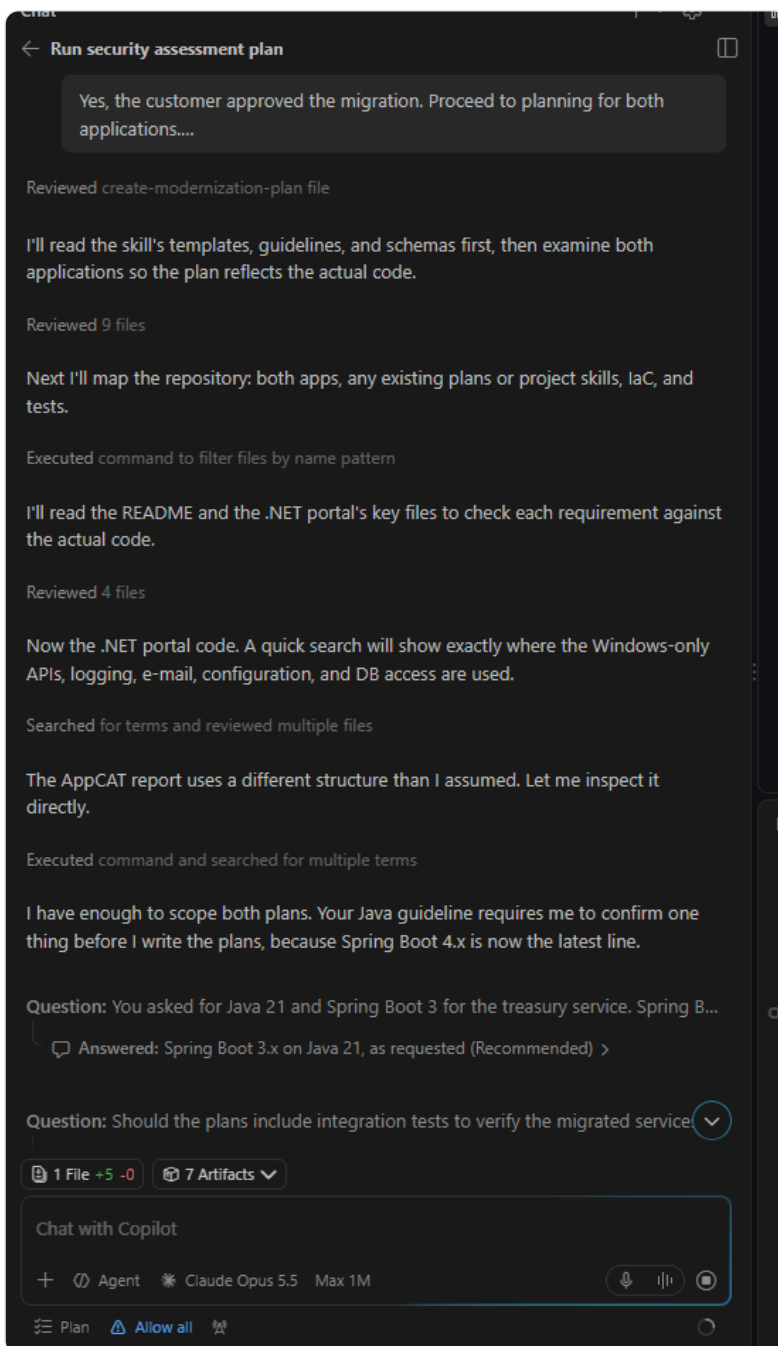
2. Treasury service (src/java/treasury-service): upgrade to Java 21 and Spring Boot 3; use Azure SQL Database with passwordless managed identity authentication; move the bank files and cash-position reports from the local disk to Azure Blob Storage with managed identity.

3. Deployment, as the last task of each plan – the portal plan runs first: deploy both applications into the existing resource group rg-finance-**team1** in swedencentral. One Linux App Service plan, plan-finance-**team1** (B1). Web apps app-portal-0720c-**team1** (.NET 10) and app-treasury-0720c-**team1** (Tomcat 10.1, Java 21, the WAR at the site root). Azure SQL server sql-finance-0720c-**team1** with Microsoft Entra-only authentication and the databases ThurayaFinance and ThurayaTreasury (Basic): create the schemas, load the demo data from the database folder and create database users for both managed identities. Storage account stfinance0720c**team1** with the containers the treasury service needs. Application Insights for the portal. Azure Communication Services Email with an Azure-managed domain. System-assigned managed identities, each with only the roles it needs – no passwords or keys. Public access is fine: this is a demo. At the end, run scripts/Invoke-SmokeTest.ps1 against both Azure URLs and show me the URLs.

Not timed yet. Expect more tasks than in the plans below, and a longer Part 5.

Copilot may ask a few more questions in a box. Choose:

Question – the wording varies	Choose
Spring Boot 3.x on Java 21, or Spring Boot 4.x on Java 25?	Spring Boot 3.x on Java 21, as requested (Recommended)
Should the plans include integration tests?	The (Recommended) option – tests with the Azure services mocked
Which Azure subscription?	The one that holds your resource group rg-finance- team1
Any other question	The (Recommended) option, if it matches the prompt you sent



Copilot reads both applications, then asks its questions in boxes – here answered with the (Recommended) option.

WHAT IT DOES

Copilot reads the findings and the code of both applications and writes one plan per application – a plan covers one language. It commits the plans; no code changes yet.

YOU SHOULD SEE

A summary of two plans, `group-finance-portal-to-azure` and `treasury-service-to-azure`, ending with something like *To start, ask me to run either plan*. If Copilot asks *The plan is ready. What would you like to do?*, choose **Review the plan first**.

Chat

← Run security assessment plan

Yes, the customer approved the migration. Proceed to planning for both applications....

There are two plans because a plan can target only one language, Java or .NET.

Group Finance Portal ([plan.md](#), [tasks.json](#))

1. Baseline: record how the portal behaves today, for Mock-mode integration tests.
2. Upgrade: move to ASP.NET Core MVC on .NET 10. This task also removes the registry, Windows identity and IIS-specific calls, and reads the treasury URL from configuration with no `localhost` fallback.
3. Database: Azure SQL Database with passwordless managed identity instead of Windows authentication.
4. E-mail: Azure Communication Services Email instead of the SMTP pickup folder.
5. Logging: OpenTelemetry to Application Insights instead of the Event Log and local log files.
6. Integration tests (Mock mode), then a CVE scan and fix.

Treasury service ([plan.md](#), [tasks.json](#))

1. Baseline: record current behaviour, for Mock-mode integration tests.
2. Upgrade: Spring Boot 3.x on Java 21, as you chose. Packaging stays a WAR for now.
3. Database: Azure SQL Database with managed identity. This removes the hard-coded password and the `localhost` connection.
4. Files: bank files and cash-position reports go to Azure Blob Storage with managed identity. No built-in migration skill matches this, so the task is driven by your own requirements.
5. Integration tests (Mock mode), then a CVE scan and fix.

Two things to know before running them:

- Portal tests may not run automatically. The execution tooling documents running the baseline and integration-test tasks for Java plans only. Portal tasks 000 and 005 may need to be run separately.
- Open questions for the deployment step, recorded in the plans:
 - Which sender domain Communication Services Email will use.
 - Whether to keep the treasury service as a WAR (Spring Boot 3 needs Tomcat 10.1 or later) or switch to an executable JAR.
 - The banking connector must read bank files from Blob Storage instead of the server folder after cut-over.
 - How the treasury API will be protected once the portal reaches it over the network.
 - The treasury service still writes its log to a local file; moving it was not requested.

To start, ask me to run either plan.

5 Files +398 -0 11 Artifacts

Chat with Copilot

+ Agent Claude Opus 5.5 Max 1M

Plan Allow all

The plans are ready: one per application, each task in one line. (This one was written before deployment was added – with prompt A or B, each plan ends with a deployment task.)

4.3 Review the plans

10 min

DO

1. Open both plans: click **plan.md** in Copilot's summary, or in the Explorer open `.github/modernize/group-finance-portal-to-azure/plan.md` and `.github/modernize/treasury-service-to-azure/plan.md`. Press **Ctrl+Shift+V** for a readable preview.
2. Check:
 - ☐ the portal plan has the upgrade to **.NET 10** before the Azure tasks;
 - ☐ there are tasks for Azure SQL, Blob Storage, Application Insights and e-mail;
 - ☐ each plan has a baseline, integration tests and a CVE task;
 - ☐ each plan ends with a **deployment** task with your names – `rg-finance-team1`, `app-portal-0720c-team1`, ...
3. At the end of each plan, **Open Questions & Questionnaire** lists what Copilot decided (`[x]`) and what is still open (`[ ]`).

Something missing? Tell Copilot in the chat, for example: *Add a task to ...*

YOU SHOULD SEE

Two plans like the ones from our test below. Each plan folder holds `plan.md` and, in `.metadata`, the task list `tasks.json` – see [What Copilot writes where](#). You run them in Part 5.

▼ The two plans from our test run

`group-finance-portal-to-azure` – 8 tasks

Task	What it does
000	Baseline – records how the portal behaves today
001	Upgrade to .NET 10 (ASP.NET Core MVC) – also removes the registry, Windows identity and IIS calls, and reads the treasury URL from configuration
002	Azure SQL Database with managed identity instead of Windows authentication
003	Azure Communication Services Email instead of the SMTP pickup folder
004	OpenTelemetry and Application Insights instead of the Event Log and log files
005	Integration tests against the baseline, with the Azure services mocked
006	CVE scan and fixes
007	Deployment – Bicep for all Azure resources, the databases with demo data, and the portal

`treasury-service-to-azure` – 7 tasks

Task	What it does
000	Baseline – records how the service behaves today
001	Spring Boot 3.x on Java 21 – still packaged as a WAR
002	Azure SQL Database with managed identity – the password and the <code>localhost</code> connection go
003	Bank files and cash-position reports to Azure Blob Storage, with managed identity
004	Integration tests against the baseline, with the Azure services mocked
005	CVE scan and fixes
006	Deployment of the treasury service, then the smoke test against both Azure URLs

Still open after the run – for real use: the bank connector must read the bank files from Blob Storage, the treasury API needs protection, and the treasury service still writes its own log to a local file.

The full plans are next to this guide: [group-finance-portal-to-azure/plan.md](#) and [treasury-service-to-azure/plan.md](#). Our run let Copilot choose its own names and region; with prompt A or B your plans use your names.

Modernize and deploy

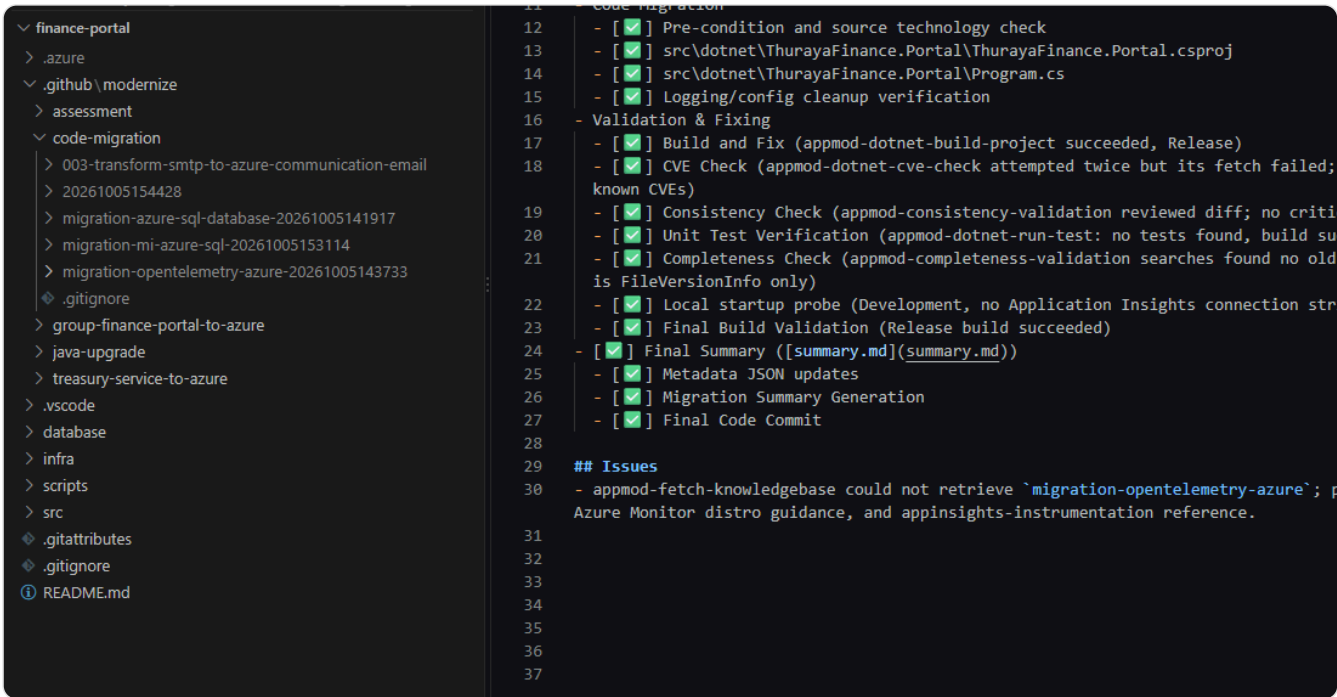
EVERYONE About 3¾ hours – Copilot works, you follow the progress

- modernize agent
- git
- .NET 10 SDK
- Java 21 + Maven (downloaded by Copilot)
- Node.js
- Azure CLI

What Copilot writes where

GitHub Copilot modernization writes everything into the folder `.github/modernize` of your repository – open it in the Explorer. This is the folder after our test run:

<code>.github/modernize/</code>	
<code>├─ assessment/</code>	Part 3 – the reports (git ignores this folder)
<code> └─ reports/report-20261005122146/</code>	one folder per report: report.json
<code> └─ engines/appcat/</code>	AppCAT logs and raw results
<code> └─ engines/security/</code>	security scan – Custom Assessment only
<code> └─ engines/facts/</code>	the 6 deep-analysis documents – Full analysis only
<code> └─ history/</code>	earlier runs
<code>├─ group-finance-portal-to-azure/</code>	Part 4 – the portal plan
<code> └─ plan.md</code>	the plan you reviewed
<code> └─ .metadata/tasks.json</code>	every task and its status
<code> └─ .metadata/summary.json</code>	every task's result: tests, risks, follow-ups
<code> └─ 000-setupBaseline-portal/</code>	baseline-summary.md
<code> └─ 005-integrationTest/</code>	verification-summary.md – the test results
<code> └─ 006-security-cve-remediation/</code>	cve-fix-summary.md
<code>├─ treasury-service-to-azure/</code>	the treasury plan – same layout
<code>├─ code-migration/</code>	Part 5 – one folder per Azure migration task
<code> └─ migration-azure-sql-database-20261005141917/</code>	plan.md, progress.md, summary.md
<code> └─ 003-transform-smtp-to-azure-communication-email/</code>	
<code> └─ migration-opentelemetry-azure-20261005143733/</code>	
<code> └─ migration-mi-azure-sql-20261005153114/</code>	
<code> └─ 20261005154428/</code>	Blob Storage – no built-in skill, so only a time stamp
<code>└─ java-upgrade/</code>	logs of the Java tools



The same folders in the VS Code Explorer during our run. The new `infra` folder holds Copilot's deployment files.

Where to see the progress of each task

Task	Where you see it	Our test
Every task	<code>.metadata/tasks.json</code> of its plan – "status" turns to <code>success</code> – and one git commit per task (Source Control view)	15 tasks, each with its own commit – plus a few commits for deployment fixes
000 Baseline	<code><plan>/000-setupBaseline-.../baseline-summary.md</code>	28 test cases for the portal, 33 for the treasury service
001 Upgrade	The chat and the task's commit – no folder of its own	.NET 10 in 14 min, Java 21 in 6 min

Task	Where you see it	Our test
Azure migrations – SQL, e-mail, monitoring, Blob	<code>code-migration/<task>-<date and time>/progress.md</code> – a checklist that fills in while Copilot works (✅ done, ⌚ running); <code>summary.md</code> when the task is done	6 to 12 min per task
Integration tests	<code><plan>/0xx-integrationTest/verification-summary.md</code>	28 of 28 and 33 of 33 passed
CVE task	<code><plan>/0xx-security-cve-remediation/cve-fix-summary.md</code>	Portal: no CVEs. Treasury: 29 fixed with patch updates.
Deployment	The chat, and the new files in <code>infra/</code> and <code>.azure/deployment-plan.md</code>	Bicep plus <code>infra/scripts/deploy.ps1</code>

5.1 Start the run

5 min, then about 3¼ hours

DO

1. In the terminal, sign in to Azure again – the deployment at the end uses this sign-in, and in our test an expired one stopped it: `az login`
2. Set **Permissions**, under the chat box, to **Allow all** ([Where you work in VS Code](#)).
3. In the same chat, send:

MESSAGE TO COPILOT

Execute both plans: first group-finance-portal-to-azure, then treasury-service-to-azure, including their deployment tasks. Commit after each task. At the end, show me the Azure URLs and the smoke test result.

4. Shows Copilot a **Review Plan** card? Click **Implement with Autopilot**.
5. Answer the question boxes Copilot may show – choose the option that matches your plan. Keep VS Code open and the laptop awake and online. Don't edit files.

1. If Copilot asks which plan: choose the **portal (.NET)** first. When it has finished, type *Execute the plan* again and choose the **treasury service (Java)**.

as IT IS DEMO SO KEEP THE LOWEST sku AND ENABLE PUBLIC ACCESS AND DEPLOY IN SWEDEN CENTRAL REGION

Inspected pre-existing todos

Those nine entries come from the first security-review attempt, which was blocked. The assessment was completed later by the parallel workers, so I'll mark them done.

Closed stale assessment todos

I've written the plan to the session folder and added the 18 todos to the session database, with dependencies.

Present plan

Plan review required

5 Files +398 -0 11 Artifacts

Review Plan

Open Full Plan

- **Order (your lab script):** run the **portal plan** first (tasks 000–006, then a new deployment task). Then run the **treasury plan** (tasks 000–005, then a deployment task).
- **Plan updates first:** add a deployment task to both repo plans. Close their open questions: Communication Services sender = Azure-managed domain; treasury on App Service with Tomcat 10.1 + Java 21, keeping the WAR; APIs public for the demo. No subscription names or IDs go into repo files.
- **Git:** new branch from modernize/dryrun, one commit per task, **no push**. Each plan's `tasks.json` and `summary.json` are updated as tasks finish.
- **Tooling:** the .NET 10 SDK is already installed. JDK 21 and Maven will be installed with the appmod tools. Baseline and Mock-mode integration tests run directly for **both** plans, which avoids the gap where the coordinator only routes these tasks for Java plans.
- **Azure (Sweden Central, new resource group, Azure CLI default subscription), Bicep in infra/ via Azure CLI:**
 - **App Service:** one Linux plan, **F1 Free**, shared by both apps. Switch to B1 only if F1 can't be created or the Java app can't run on it.
 - **Web apps:** portal on .NET 10, treasury on Tomcat 10.1 / Java 21. Each has its own managed identity.
 - **Azure SQL:** server with Microsoft Entra-only sign-in and two **Basic** databases.
 - **Storage:** **Standard_LRS** account with `bank-files` and `reports` containers.
 - **E-mail:** Communication Services Email with an Azure-managed (free) domain.
 - **Monitoring:** Log Analytics and Application Insights.
- **Public access:** public network access is on (SQL firewall: Azure services plus your IP). Anonymous blob access and storage keys stay **off**; managed identity only.
- **Data:** create the schemas and load one generated demo dataset into both databases with a small script (`sqlcmd` isn't installed). Create database users and role assignments for both managed identities.
- **Verification:**

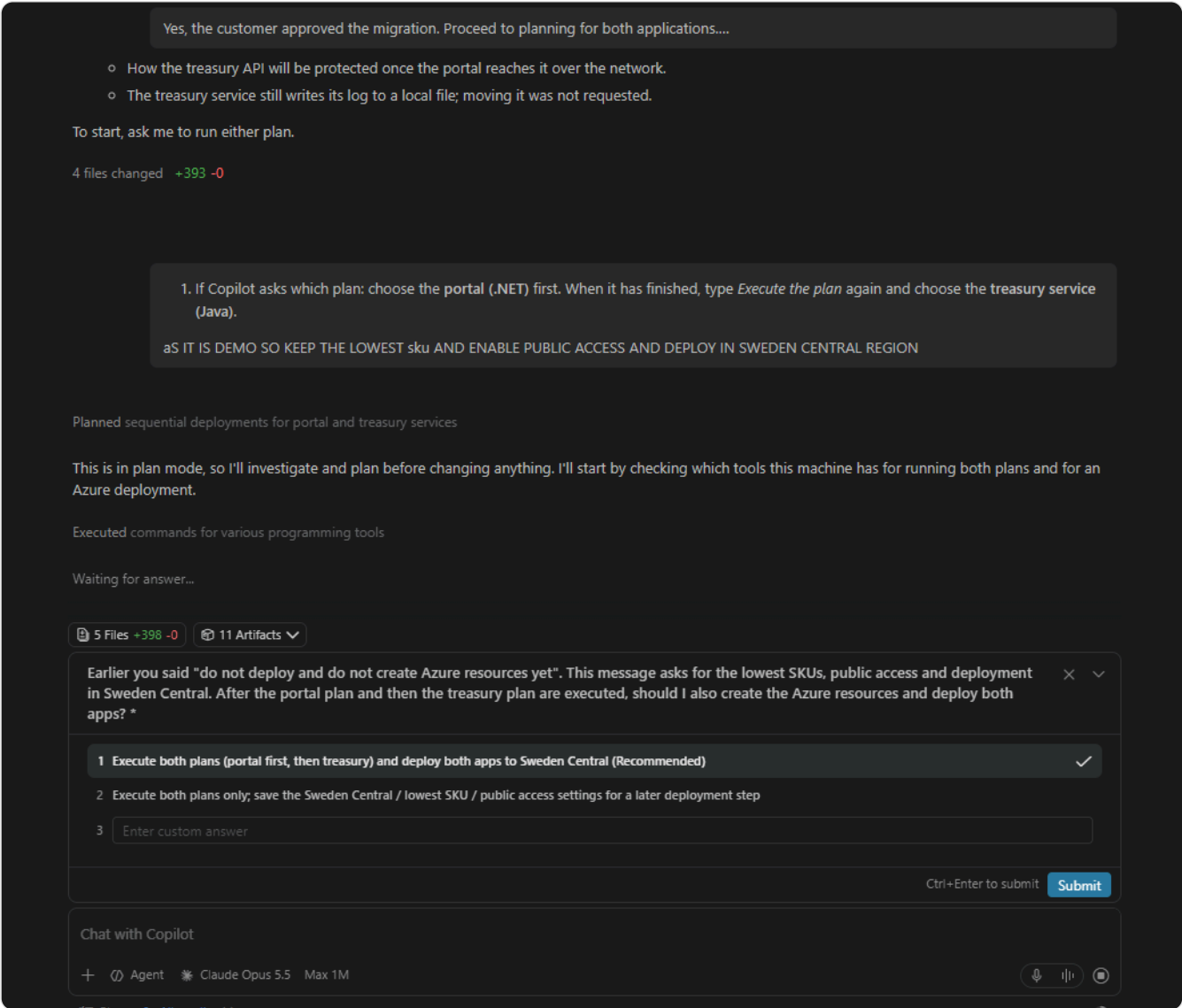
Implement with Autopilot Reject

Chat with Copilot

+ Agent Claude Opus 5.5 Max 1M

Plan Allow all

In *Plan* mode Copilot first shows its run plan – click **Implement with Autopilot**.



A question during the run: our first plan request said *don't deploy*, so Copilot asked. With prompt A or B the deployment is already in the plans.

WHAT IT DOES – FOR EVERY TASK

Copilot changes the code, builds it and runs the tests, fixes what breaks, checks the result against the original behaviour, and commits – one git commit per task on your branch. The status bar shows which branch. At the end it creates the Azure resources, loads the demo data into Azure SQL, deploys both apps and runs the smoke test against them.

The run stopped, or a task shows *Started* instead of *done*? Usually the network dropped. Type *Continue executing the plan* – Copilot picks up where it stopped.

YOU SHOULD SEE

Commits appearing one by one. Our test, 5 October 2026:

Time	What happened
13:51	Plans committed
14:03 – 15:04	Portal: baseline, .NET 10, Azure SQL, e-mail, OpenTelemetry, integration tests, CVE task – 7 commits
15:14	Portal deployment task: Bicep for all resources and the database script
15:23 – 16:17	Treasury: baseline, Java 21, Azure SQL, Blob Storage, integration tests, CVE task – 6 commits
16:20 – 16:24	Deployment script, updated README
16:36 – 17:38	Deployment: Azure sign-in renewed, a start-up bug fixed, region changed – both apps live

5.2 Follow the progress

while it runs

DO – IN THE EXPLORER

1. Open `.github/modernize/code-migration`. A new folder appears for each Azure migration task. Open its `progress.md` – it fills in while Copilot works.
2. Open `.github/modernize/group-finance-portal-to-azure/.metadata/tasks.json`: each finished task shows `"status": "success"`.
3. Open the **Source Control** view: one commit per finished task.

The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays a file tree with folders like `plan.md`, `post-migration-plan.md`, `progress.md`, and `summary.md` under `.github/modernize/code-migration`. The main editor area shows the content of a `progress.md` file for the task `migration-opentelemetry-azure-20261005143733`. The file content includes a session ID, workspace path, language (dotnet), and a detailed progress log with checkboxes indicating the completion of various steps such as 'Migration Plan Generation', 'Version Control Setup', 'Code Migration' (including pre-condition checks, build and fix, CVE check, consistency check, unit test verification, and completeness check), 'Validation & Fixing', 'Local startup probe', 'Final Build Validation', 'Final Summary', 'Metadata JSON updates', 'Migration Summary Generation', and 'Final Code Commit'. An 'Issues' section at the bottom notes a failure to retrieve migration-opentelemetry-azure data but states the workflow will proceed.

`progress.md` of the OpenTelemetry task: every step ticked – code changes, build, CVE, consistency and completeness checks, startup test and commit.

YOU SHOULD SEE

A `progress.md` like the one above for each Azure migration task, and a `summary.md` next to it when the task is done – with a before-and-after table and the build, test and CVE results. See [What Copilot writes where](#) for every other report.

5.3 Read the final summary

5 min

When both plans are done, Copilot ends with a summary in the chat. Ours, shortened:

Task completed: both modernization plans ran to completion, portal first, and both apps are live on Azure. The end-to-end smoke test passes 15/15 against the deployed apps, and the portal's `/health` reports `Healthy`.

Portal: ASP.NET Core MVC on .NET 10 with the Windows-only APIs removed. Azure SQL with managed identity, Communication Services Email, OpenTelemetry to Application Insights. 28/28 integration tests pass; no CVEs.

Treasury service: Spring Boot 3.5.16 on Java 21, still a WAR on Tomcat 10.1. Azure SQL with managed identity; bank files and reports in Blob Storage. 12 unit and 33 integration tests pass; 29 CVEs fixed with patch-level updates.

Every app-to-service connection uses managed identities, each with only the role it needs. Nothing stores a password or key. Everything is committed, not pushed. To redeploy: `infra/scripts/deploy.ps1`.

Before any real use: the bank connector must read the bank files from the `bank-files` container, the treasury API should be protected, and the treasury log still goes to a local file.

Why B1 in prompts A and B? Our test used the free F1 tier. The first deployment showed an upgrade bug: the portal's `assets` folder was missing from the published app, so it crash-looped. On F1 that used up the plan's hourly restart allowance and locked both apps. Copilot fixed the bug and redeployed in another region. B1 has no such limit.

YOU SHOULD SEE

The two Azure URLs and the smoke test result – 15 × `[PASS]`. You check them yourself in Part 6.

5.4 See what changed

10 min

DO – IN THE TERMINAL

```
git log --oneline
dotnet build src/dotnet
dotnet test src/dotnet
```

Then open a few commits in the **Source Control** view. This is what Copilot changed:

Before	After
Portal (<code>src/dotnet</code>)	
.NET Framework 4.8, <code>Global.asax</code> , <code>Web.config</code>	.NET 10, <code>Program.cs</code> , <code>appsettings.json</code>
Windows authentication to SQL Server	Azure SQL with managed identity – no password
E-mail into a local SMTP folder	Azure Communication Services Email
Event Log and local log files	OpenTelemetry → Application Insights
Treasury URL written in the code	Treasury URL from configuration
Treasury service (<code>src/java/treasury-service</code>)	
Java 8, Spring Boot 2.7 (<code>javax</code>)	Java 21, Spring Boot 3 (<code>jakarta</code>)
SQL password in <code>application.properties</code>	Azure SQL with managed identity – password gone
Bank files and reports on the local disk	Blob Storage with managed identity
Outdated libraries	Patched versions (CVE task)
New	
–	<code>infra/</code> : Bicep for all Azure resources, the database script and <code>infra/scripts/deploy.ps1</code>

YOU SHOULD SEE

- One commit per task, for both plans, and `Build succeeded` and `Passed!`

Worth showing: Copilot finds and fixes its own mistakes. In an earlier test, its tests caught a bug from the .NET upgrade (error 500 instead of 403). In this run, the first deployment showed the missing `assets` folder – Copilot fixed the publish step and redeployed.

6

Test on Azure

EVERYONE About 20 minutes · use the VS Code terminal

Invoke-SmokeTest.ps1

Azure portal

Azure CLI

Copilot deployed both apps in Part 5. Your Azure names: resource group `rg-finance-team1`, portal `app-portal-0720c-team1`, treasury service `app-treasury-0720c-team1`, SQL server `sql-finance-0720c-team1`.

6.1 Run the test on Azure

5 min

DO

```
$portal = 'https://app-portal-0720c-team1.azurewebsites.net'
$treasury = 'https://app-treasury-0720c-team1.azurewebsites.net'
./scripts/Invoke-SmokeTest.ps1 -PortalUrl $portal -TreasuryUrl $treasury
```

THE TWO ADDRESSES

The same two parts as in 3.2 – now on Azure: `$portal` is the portal Copilot deployed, `$treasury` the treasury service. Copilot used other names? Take the two URLs from its final summary (5.3).

WHAT IT DOES

Exactly the same test as in 3.2. If it passes on Azure too, the migrated app behaves like the original.

A check fails, or the first request times out? Run the test again after a minute – the first request wakes the apps up. Still failing? Paste the test output into the chat and let Copilot fix it.

YOU SHOULD SEE

15 × `[PASS]` and `SMOKE_TEST_PASSED` – as in our test.

6.2 Look at it on Azure

10 min

DO

1. Open the portal on Azure: `https://app-portal-0720c-team1.azurewebsites.net` – the same invoices and figures as on the VM. User menu → **About**.
2. Open `https://app-portal-0720c-team1.azurewebsites.net/health`.
3. In the Azure portal, open the resource group `rg-finance-team1`: the two web apps, the SQL server with two databases, the storage account, Application Insights and Communication Services. On a web app, **Identity** shows its managed identity.

YOU SHOULD SEE

- **About** shows Kestrel / ASP.NET Core on .NET 10, an Azure SQL database (*SQL Azure*) and Java 21 – instead of IIS, .NET Framework 4.8, SQL Server Express and Java 8.
- `/health` reports `Healthy`.
- No passwords or keys in any app setting – only server names, database names and endpoints.
- The same kinds of resources as in [The result](#) below, with your names.

The result: the whole application on Azure

This is where the migration ends: everything that ran on the on-premises server now runs on Azure. The resource group from our test run – 13 resources, created by Copilot's deployment:

rg-thuraya-demo-frc

Resource group

Generate Bicep code to duplicate this resource group.

How to manage changes with deployment tools?

+1

Search

Create

Manage view

Delete resource group

Refresh

Export to CSV

Open query

Assign tags

Overview

Activity log

Access control (IAM)

Tags

Analyze (Preview)

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

Subscription (move) :

Deployments : 2 Succeeded

Subscription ID :

Location : France Central

Tags (edit) : azd-env-name : thuraya-demo-frc workload : althuraya-group-finance Show more (2)

Resources

Recommendations

Filter for any field...

Type equals all

Location equals all

+ Add filter

☐	Name ↑	Type	Location
☐	acs-thuraya-demo-frc-yb5y4d5uekb4e	Communication Service	Global
☐	app-thuraya-demo-frc-portal-yb5y4d5uekb4e	App Service	France Central
☐	app-thuraya-demo-frc-treasury-yb5y4d5uekb4e	App Service	France Central
☐	appi-thuraya-demo-frc-yb5y4d5uekb4e	Application Insights	France Central
☐	asp-thuraya-demo-frc-yb5y4d5uekb4e	App Service plan	France Central
☐	AzureManagedDomain (ecs-thuraya-demo-frc-yb5y4d5uekb4e/AzureMa	Email Communication Services Do...	Global
☐	ecs-thuraya-demo-frc-yb5y4d5uekb4e	Email Communication Service	Global
☐	Failure Anomalies - appi-thuraya-demo-frc-yb5y4d5uekb4e	Smart detector alert rule	Global
☐	log-thuraya-demo-frc-yb5y4d5uekb4e	Log Analytics workspace	France Central
☐	sql-thuraya-demo-frc-yb5y4d5uekb4e	SQL server	France Central
☐	stthurayadeyb5y4d5uekb4e	Storage account	France Central
☐	ThurayaFinance (sql-thuraya-demo-frc-yb5y4d5uekb4e/ThurayaFinance)	SQL database	France Central
☐	ThurayaTreasury (sql-thuraya-demo-frc-yb5y4d5uekb4e/ThurayaTreasury	SQL database	France Central

Showing 1 - 13 of 13. Display count: 200

New Brows

Add or remove favorites by pressing Ctrl+Shift+F

Our run, 5 October 2026 (subscription hidden). Copilot chose the names (thuraya-demo-frc-...) and, after the F1 problem, France Central. With prompt A or B your resource group rg-finance-team1 shows the same kinds of resources with your names.

On the server (before)	On Azure (now)	In the resource group
IIS, ASP.NET MVC on .NET Framework 4.8	Portal on App Service – Linux, .NET 10	app-...-portal-... on the plan asp-...
Tomcat 9, Java 8, Spring Boot 2.7	Treasury service on App Service – Tomcat 10.1, Java 21, Spring Boot 3	app-...-treasury-...
SQL Server Express – two databases, Windows authentication and a SQL password	Azure SQL Database with managed identities – no passwords	sql-... with ThurayaFinance and ThurayaTreasury
Bank files and reports on the server's disk	Blob Storage	st...
E-mail through an SMTP pickup folder	Azure Communication Services Email	acs-..., ecs-... and its AzureManagedDomain
Windows Event Log and log files	OpenTelemetry → Application Insights	appi-... with log-..., and the Failure Anomalies alert rule that Azure adds to it

Done end to end by the GitHub Copilot modernization agent. It assessed the code, wrote the plans, upgraded and migrated both applications, tested them, wrote the Bicep and the deployment script, created these resources, loaded the demo data, deployed both apps and ran the smoke test. People made the decisions: approve the migration, choose the scope, review the plans, answer Copilot's questions and sign in to Azure. Nobody edited the code or created a resource by hand.

6.3 Clean up

2 min

Security: the treasury service's API has no sign-in – on-premises it relied on the server's network. On Azure it is reachable from the internet, so don't leave the deployment running.

DO

```
az group delete -n rg-finance-team1 --yes --no-wait
```

YOU SHOULD SEE

The command returns at once; Azure deletes your resources in the background. Your branch keeps the full story – code, plans, reports and the deployment script: `git log --oneline`.

7 Run the day

PRESENTER Agenda, customer demo, troubleshooting and clean-up

Agenda

Time	Part	Your focus
0:00	Intro (15 min)	The story: the CFO wants Azure; humans stay in the loop
0:15	3 – Assess (30 min)	Help with tool downloads; point out the mandatory findings
0:45	4 – Decide and plan (30 min)	Play the customer: ask about risks and effort, then say yes
1:15	5 – Modernize and deploy (about 3¾ hours, lunch included)	Check that every team chose prompt A, signed in with <code>az login</code> and set Allow all; keep teams from editing code; collect "surprises"
5:00	6 – Test on Azure (20 min)	Smoke test on Azure; a look at the managed identities
5:20	Read-out (20 min)	What Copilot did, where teams stepped in, open risks

Customer demo (30–40 minutes)

Part 5 takes hours, so prepare the day before: run Parts 2–6 yourself and keep the finished branch and the Azure deployment. Show live only the moments where the customer decides.

- Today (3 min):** the portal on the VM, approve an invoice, **About** – IIS, .NET Framework 4.8, SQL Server, Tomcat, Java 8.
- The code (3 min):** `Web.config` (Windows authentication, SMTP folder, log file), `application.properties` (password), `BankFileWriter.java` (local disk).
- Assess – live (7 min):** 3.3, then the report with **Target Service** set to Azure App Service and its mandatory findings. Ran the full assessment (3.4, optional) the day before? Show its extra tabs, such as **Architecture**.
- The decision (3 min):** when Copilot asks its next question, turn to the customer: "Do you want to move it to Azure?"
- Plan (5 min):** explain scope A and B, send prompt A live. Planning takes about 15 minutes, so continue with the plans you prepared – or the sample plans in 4.3: ordered tasks, each tied to findings, ending with the deployment.
- The run (7 min):** your prepared branch – `git log` with one commit per task, a `progress.md` in `.github/modernize/code-migration`, a few diffs, and the bugs Copilot found and fixed itself.
- On Azure (5 min):** the resource group Copilot created ([The result](#)), the deployed portal, **About**, `/health`, the managed identities, and the smoke test passing.
- Close (2 min):** assessment → decision → plan → reviewed changes → Azure, for both stacks – end to end by GitHub Copilot modernization.

Troubleshooting

Problem	Fix
The portal or Remote Desktop times out	The network changed – run the first four lines of 1.5 from the network you are on. Some company networks use several internet addresses: use the venue Wi-Fi or a phone hotspot instead.
Portal shows HTTP 500 or 503 right after the VM started	Wait 3–5 minutes – SQL Server and Tomcat are still starting.
Azure CLI: AADSTS70043 or "token expired"	<code>az login</code> , then run the command again.
GitHub Copilot modernization shows only Open Folder , or no Migrate to Azure button	No folder is open: File → Open Folder... → <code>C:\Lab\finance-portal</code> . Then click QuickStart to open it (3.3).
The chat shows <i>Agent, Ask or Plan</i> instead of modernize , or you lost the chat	Click the agent picker at the bottom of the chat box and choose modernize . To find the chat: click ← at the top of the Chat panel and open <i>Migrate application to Azure</i> .
Copilot shows a question box, but you want to send your own text	Click Enter custom answer , paste the text, press Ctrl+Enter .
The report shows <i>No Dockerfile found</i> and other container findings	Set Target Service to Azure App Service (3.4).
Custom Assessment with Full analysis stays at 99%	The deep-analysis documents haven't arrived. The lab doesn't need them: click Stop Running – the findings stay.
Lost the VM password	<code>az vm user update -g rg-contoso-onprem-swc -n vm-contoso-web01 -u contosoadmin -p <new-password></code>
Assessment: <i>Msbuild was not found / Report file is not generated</i>	Install the Build Tools (2.1), restart VS Code, assess again.

Problem	Fix
Assessment: <i>Failed to install .NET AppCAT tool</i> (nuget.org blocked by a company proxy)	Install it from your company's approved NuGet feed, then assess again: <code>dotnet tool install --tool-path "\$env:LOCALAPPDATA\Microsoft\VisualStudio\AppModernizationExtension\Tools" dotnet-appcat</code>
The plan has no .NET 10 upgrade task	Use the exact 4.2 prompt – it asks for .NET 10.
The run stopped, or a task shows <i>Started</i>	Usually a network drop. Type <i>Continue executing the plan.</i>
The deployment waits for an Azure sign-in, or Azure CLI says <i>AADSTS70043</i>	Run <code>az login</code> in the terminal, then tell Copilot: <i>I signed in again – continue.</i>
Copilot asks whether it should deploy	Your plan request didn't include deployment. Choose the option that executes both plans and deploys.
The portal on Azure keeps restarting; its log mentions the missing <code>assets</code> folder	The upgrade left <code>assets</code> out of the published app. Tell Copilot: <i>The portal's publish output is missing the assets folder – include it and redeploy.</i> In our test Copilot found and fixed this itself.
F1 (free) App Service: apps stopped after restarts, or no F1 quota in the region	Use B1 as in prompts A and B. F1 limits CPU time and restarts.
Copilot used other names than in your prompt	Use the URLs from its final summary (5.3) in 6.1.
App on Azure: <i>Login failed for user '<token-identified principal>'</i>	A managed identity has no database user. Tell Copilot the error – it creates the user and restarts the app.
Portal on Azure shows no data; log: <i>No such host is known</i>	The database setting still has a placeholder. Tell Copilot: <i>Set the portal's connection string ConnectionStrings__ThurayaFinanceDb to my Azure SQL server.</i>
No data in Application	Tell Copilot: <i>Set the App Service setting ApplicationInsights__ConnectionString for the portal.</i>

Reset and clean up

```
./onprem-vm/Update-LabApp.ps1 -ResourceGroup rg-contoso-onprem-swc -ResetDemoData # fresh demo data before the next
session
az vm deallocate -g rg-contoso-onprem-swc -n vm-contoso-web01 # pause the VM between events
(stops compute costs)
az group delete -n rg-finance-team1 --yes --no-wait # a team's Azure resources - repeat
for team2, team3, ...
./onprem-vm/Remove-Lab.ps1 -ResourceGroup rg-contoso-onprem-swc # only when you no longer need the
lab VM
```

`Update-LabApp.ps1` starts a paused VM automatically. The lab VM costs about USD 35 per month while it runs.

What was tested

5 October 2026 – the whole VS Code path, end to end by the GitHub Copilot modernization agent (version 1.24.0, Claude Opus 5.5), in a clone of the lab repository:

- QuickStart → **Migrate to Azure** → standard assessment: 5 findings in the treasury service, in a few minutes.
- Custom Assessment with Full analysis and Security: 14 issues; the six documents didn't arrive and it was cancelled at 99%.
- Planning → two plans; deployment added before the run (8 and 7 tasks, 4.3).
- Run with **Implement with Autopilot**: all 15 tasks succeeded, one commit each. Portal 28/28 integration tests; treasury 12 unit and 33 integration tests; 29 CVEs fixed.
- Copilot's deployment: Bicep, Azure SQL with demo data, both apps – 13 resources ([The result](#)), smoke test 15/15 on Azure, `/health` Healthy. Plans to live in about 3¾ hours, including an F1 restart lock-out, a region change and an expired Azure sign-in.

2 October 2026 – the same agents through the Modernize CLI, in copies of the code:

- Lab deployment, smoke test (15/15) and data check on the VM.
- Assessment of both applications: 15 portal issues and 11 treasury findings.
- Code changes: all tasks succeeded – portal in about 105 minutes, treasury in 72 minutes.
- Data copy from the VM to Azure SQL with `scripts/Copy-DatabasesToAzureSql.ps1`: 19 tables / 1,902 rows and 7 tables / 279 rows identical – the alternative to the demo data, if you want the server's real data ([04-deploy-and-verify.md](#)).

Not tested yet – rehearse once before the first event: prompt A and B word for word – with your names, B1 and an existing resource group – and prompt B's extra security work.

This guide is set up for the lab VM `contoso-onprem-0720c` in resource group `rg-contoso-onprem-swc`. For a lab in another subscription or resource group, replace `0720c` and `rg-contoso-onprem-swc` in this file.

Application code: [zhshah/AIThuraya-App-Modernization-Lab](#). Lab kit – this guide, the lab scripts and the application: [zhshah/AIThuraya-App-Modernization-LabKit](#) (presenters). Keep the `images` and `sample-plans` folders next to this file – the screenshots and sample plans live there. Presenters: the Markdown files next to this guide ([README.md](#) and the modules) are the detailed reference, including optional alternatives such as the Modernize CLI.

All organisations, people, accounts and figures in the demo application are fictitious.